

Hardware Diagnostics Root Cause & Fix Plan

Goal Description

During hardware testing on **COM8**, several critical issues were observed:

1. **Pen lift (Option 1)** worked because it dispatched only a single isolated command.
2. **20mm Square (Option 4)** only completed 3/4 of its perimeter and failed on the 4th side.
3. **Phrase Plotting (Option 5)** crashed completely.
4. Several other diagnostic menu items failed to execute as expected.

By performing live non-destructive hardware inspection on **COM8**, we extracted the exact GRBL EEPROM settings, parser state, and buffer behaviors. This identified three major root causes:

Root Cause 1: The CRLF Newline Desynchronization Bug (Fatal)

- In `grbl_controller.py`, commands were formatted with `\r\n` (`formatted = gcode_line.strip() + "\r\n"`).
- In GRBL 0.9i, **both** `\r` (CR) and `\n` (LF) are treated as distinct line terminators.
- Consequently, for every single G-code line sent, GRBL returned **two** `ok` responses (`ok\r\nok\r\n`): one for the command, and one for the empty newline.
- `send_blocking()` returned immediately upon reading the first `ok`, leaving the second "ghost" `ok` stranded in the serial receive buffer.
- **Why Option 4 drew 3/4 of a square:** With each edge of the square, ghost `ok` responses accumulated in the serial buffer. By the 4th side, the host read old `ok` tokens before GRBL even received or processed the 4th move, desynchronizing the motion queue and dropping commands.
- **Why Option 5 crashed:** A complete word contains dozens of rapid moves and draws. The ghost `ok` flood overwhelmed GRBL's 127-byte RX buffer, causing a serial buffer overflow, corrupted coordinates, and a fatal communications timeout crash.

Root Cause 2: Travel Limits & Out-of-Bounds Movement

- Querying GRBL EEPROM revealed:
- `$130 = 200.000` (X-axis max physical travel: **200 mm**)
- `$131 = 200.000` (Y-axis max physical travel: **200 mm**)
- `$110 = 2000.000` (X max velocity: 2000 mm/min)
- `$111 = 2000.000` (Y max velocity: 2000 mm/min)
- `$100 = 81.800` & `$101 = 81.800` (Resolution: **81.8 steps/mm**, corresponding to 20-tooth GT2 pulleys)
- In `config.json`, the default baseline was set to `start_y = 270.0 mm` and `page_height = 290.0 mm`.
- When Option 5 ran, `reset_origin()` commanded `G0 X15.00 Y270.00`. Commanded `$Y=270$` is **70 mm beyond the machine's physical travel limit of 200 mm**, causing the carriage to jam against the physical rails or trigger alarm lock.

Root Cause 3: Stray Work Coordinate Offsets (G54)

- Querying GRBL parameter memory (`$#`) revealed an active G54 work offset: `[G54: -30.000, 60.000, 0.000]`.
- Because absolute positioning (`G90`) moves relative to the active work coordinate system, this offset shifted all motions by -30 mm in X and $+60\text{ mm}$ in Y, skewing the origin and causing travel truncation.

User Review Required

IMPORTANT

Workspace Envelope: Your GRBL configuration limits travel to **200 mm $\times$ 200 mm**. We will update `config.json` so that the default writing envelope fits comfortably within your machine (`page_width: 180.0` , `page_height: 180.0` , `start_y: 170.0` , `margin_left: 15.0`).

Feed Rate Limit: We will adjust `rapid_feed` from 2500 to **1800 mm/min** so it stays safely within your GRBL maximum rate of 2000 mm/min.

Coordinate Zeroing: We will add automatic G54 offset clearing (`G10 L2 P1 X0 Y0 Z0`) and work coordinate zeroing (`G92 X0 Y0`) upon startup so the current physical position of the pen is always guaranteed to be $(0, 0)$.

Open Questions

NOTE

- What are the physical bed dimensions of your machine? Is 200mm $\times$ 200mm the actual physical travel of your gantry rails?
- For microstepping resolution: GRBL is currently set to `$100=81.8` and `$101=81.8` (standard for 20-tooth pulleys). Does your machine have 16-tooth pulleys (100 steps/mm) or 20-tooth pulleys (80–81.8 steps/mm)?

Proposed Changes

We will modify the core controller, layout compiler, test utility, and configuration, and add persistent logging.

Motion Controller & Logging Layer

[MODIFY] `grbl_controller.py`

- Change line termination from `\r\n` to strictly `\n` to eliminate duplicate `ok` responses.

- Implement robust input buffer flushing before sending commands.
- Add persistent logging via Python `logging` to a dedicated `hardware.log` file, recording:
 - Timestamp
 - Raw sent command bytes
 - Every incoming line from GRBL
 - Response latencies and timeout events
- Reset G54 coordinate offsets during `initialize_motion_state()` with `G10 L2 P1 X0 Y0 Z0`.
- Update communication timeout logic so long moves do not throw premature `TimeoutError`.

[MODIFY] `config.json`

Update machine parameters to match actual GRBL capabilities:

```
{
  "serial": {
    "port": "COM8",
    "auto_detect": true,
    "baud_rate": 115200,
    "boot_delay_seconds": 2.5,
    "timeout": 5.0
  },
  "motion": {
    "cmd_pen_up": "M03 S90",
    "cmd_pen_down": "M03 S0",
    "servo_dwell": 0.35,
    "rapid_feed": 1800,
    "draw_feed": 800
  },
  "geometry": {
    "page_width": 180.0,
    "page_height": 180.0,
    "margin_left": 15.0,
    "start_y": 165.0,
    "line_spacing": 9.5,
    "font_scale": 0.28
  },
  "speech": {
    "engine": "google",
    "ambient_duration": 1.0,
    "phrase_timeout": 8.0
  }
}
```

Layout Compiler & Boundaries

[MODIFY] `vector_compiler.py`

- Respect the new travel boundary `($Y \le 180\text{ mm})$, ($X \le 180\text{ mm})$.`
- Add travel bounds safety validation before dispatching moves, preventing any G-code command from exceeding physical workspace limits.

Diagnostic Test Suite

[MODIFY] `test_hardware.py`

- Fix Option 6 (GRBL `$$` settings readout) to read and display all 32 configuration parameters cleanly without shell variable interference.
- Fix Option 4 (Calibration square): ensure clean 4-sided closed square with strict single-token ping-pong flow control.
- Fix Option 5 (Test phrase): plot within safe `$Y \le 165\text{ mm}` bounds.
- Add Option for real-time position reporting (`?` query) showing current `$X/Y` coordinates.
- Save full diagnostic run log to `hardware_test.log`.

Verification Plan

Phase 1: Communication & Flow-Control Validation

1. Send a sequence of 20 rapid G-code commands via test script.
2. Verify that **exactly one** `ok` is received per command and zero leftover bytes remain in the serial buffer.
3. Check `hardware.log` to confirm zero dropped packets.

Phase 2: Square Calibration (Option 4)

1. Run `test_hardware.py` -> Option 4.
2. Verify all 4 sides of the 20mm square draw completely, close precisely at the starting point, and lift the pen without dragging.
3. Measure physical square with a ruler to verify dimension accuracy (evaluating 81.8 steps/mm).

Phase 3: Single-Stroke Phrase Plotting (Option 5)

1. Run `test_hardware.py` -> Option 5.
2. Verify carriage moves to `$(15, 165)`, lowers pen, and writes `"TEST 123"` with clean pen lifts and zero motor stalls or crashes.

Phase 4: Full Dictation Pipeline (`voice_scribe_engine.py`)

1. Run `voice_scribe_engine.py`.
2. Speak test phrase $\rightarrow$ say `"OK"` $\rightarrow$ verify text draws smoothly on paper.